

ASSIGNMENT-7

Task – 1: Git Fundamentals

SOLUTION-

Introduction

Git is a distributed version control system used to track changes in files and source code. It allows developers to manage different versions of a project, collaborate with others, and maintain a complete history of changes. **GitHub Desktop** provides a graphical user interface (GUI) to use Git without remembering terminal commands.

1. Install Git and GitHub Desktop

- Download and install **Git** on your computer.
- Install **GitHub Desktop**.
- Sign in with your GitHub account.

Purpose:

Git performs version control, while GitHub Desktop provides an easy graphical interface to use Git features.

2. Configure Git

After signing in, configure your Git identity.

- **Name:** Praveen Singh
- **Email:** your_email@example.com

Purpose:

The configured name and email are attached to every commit, helping identify who made each change.

3. Create a New Repository

- Open **GitHub Desktop**.
- Click **File → New Repository**.

- Enter the repository name (for example, Git_Practice).
- Choose a local folder.
- Click **Create Repository**.

Purpose:

A repository is the main folder where Git stores project files and tracks their changes.

4. Create Three Files

Inside the repository, create three files:

- file1.txt
- file2.txt
- file3.txt

These files will automatically appear in the **Changes** section of GitHub Desktop.

5. First Commit

- Enter a commit message, for example: **First Commit**.
- Click **Commit to main**.

Purpose:

A commit saves the current version of the project and records it in the Git history.

6. Make Changes

Open file1.txt and add some text, such as:

Git Practice

Save the file.

GitHub Desktop automatically detects the modification and displays it in the **Changes** tab.

7. Stage and Commit Changes

GitHub Desktop automatically stages the changed file when selected.

- Enter the commit message: **Updated file1**
- Click **Commit to main**

Purpose:

This creates another version of the project containing the latest changes.

8. View Commit History

Click the **History** tab.

It displays:

- First Commit
- Updated file1

Purpose:

The History tab shows all commits made in the repository along with their messages and timestamps.

9. Undo a Change

If you edit a file but do not want to keep the changes:

- Open the **Changes** tab.
- Right-click the modified file.
- Select **Discard Changes**.

Purpose:

This restores the file to its last committed version.

10. View File Differences (Diff)

Select a modified file in the **Changes** tab.

GitHub Desktop highlights:

- Added lines (green)
- Deleted lines (red)

Purpose:

The Diff view helps compare the current file with the previous committed version.

Summary

Feature	Description
Repository	Stores project files and Git history
Commit	Saves a snapshot of the project
Changes	Displays modified files
History	Shows all previous commits
Discard Changes	Restores the last committed version
Diff	Compares current changes with the previous version

Task – 2: GitHub Repository and Branches-**SOLUTION-****Introduction**

A **GitHub repository (repo)** is an online storage location where project files and their complete version history are stored. GitHub allows developers to collaborate by creating branches, making changes, reviewing code through Pull Requests (PRs), and automating tasks using GitHub Actions.

1. Create a GitHub Repository**Steps**

- Sign in to your GitHub account.
- Click **New Repository**.
- Enter the repository name:

devops-practice

- Select **Public** or **Private**.
- Click **Create Repository**.

Purpose

A repository stores project files, commit history, branches, and collaboration features in one place.

2. Push Local Commits

Open **GitHub Desktop**.

- Open the local repository.
- Commit any pending changes.
- Click **Push Origin**.

Purpose

Pushing uploads local commits from the computer to the GitHub repository, making them available online.

3. Create a New Branch

Steps

- Click the **Current Branch** menu.
- Select **New Branch**.
- Enter the branch name:

feature/update-readme

- Click **Create Branch**.

Purpose

A branch creates an independent workspace where new features or changes can be developed without affecting the main branch.

4. Make Changes

- Open the **README.md** file.
- Add some text describing the project.
- Save the file.

GitHub Desktop automatically detects the modification.

5. Commit and Push the Branch

- Enter a commit message.

Example:

Updated README file

- Click **Commit to feature/update-readme**.
- Click **Push Origin**.

Purpose

The updated branch is uploaded to GitHub.

6. Open a Pull Request (PR)

Steps

- Open the repository on GitHub.
- Click **Compare & Pull Request**.
- Review the changes.
- Click **Create Pull Request**.

Purpose

A Pull Request allows team members to review changes before merging them into the main branch.

7. Merge the Pull Request

- Open the Pull Request.

- Click **Merge Pull Request**.
- Click **Confirm Merge**.

Purpose

The changes from the feature branch are merged into the **main** branch.

8. Delete the Branch

After merging:

- Click **Delete Branch**.

Purpose

Deleting the feature branch keeps the repository clean after the work has been completed.

9. Set Up GitHub Actions

Steps

- Open the repository on GitHub.
- Click the **Actions** tab.
- Create a new workflow.
- Name the workflow:

Continuous Integration (CI) Workflow

Create the workflow file:

`.github/workflows/ci.yml`

The workflow runs the command:

`echo "CI passed"`

on every push.

Purpose

GitHub Actions automatically performs tasks whenever code is pushed to the repository. This process is known as **Continuous Integration (CI)**.

Task – 3: Git Branching Strategy – GitFlow-

SOLUTION-

Introduction

GitFlow is a popular Git branching strategy introduced by Vincent Driessen. It provides a structured workflow for managing software development by separating ongoing development, new features, releases, and bug fixes into different branches. GitFlow helps teams collaborate efficiently while keeping the production code stable.

The main branches used in GitFlow are:

- **main – Stores the stable and production-ready version of the project.**
- **develop – Contains the latest development changes before release.**
- **feature – Used to develop new features independently.**

1. Main Branch

The main branch is the primary branch of the repository. It always contains stable, tested, and production-ready code. Users or customers receive the software from this branch.

Purpose:

- **Stores the final version of the project.**
- **Only tested code is merged into this branch.**
- **Each release is identified using a version tag.**

2. Develop Branch

The develop branch acts as the integration branch where developers combine completed features before releasing them.

Create the develop branch

git branch develop

git checkout develop

Purpose:

- Combines all completed features.
- Used for testing before release.
- Acts as the parent branch for all feature branches.

3. Feature Branch

A feature branch is created from the develop branch whenever a new feature needs to be developed.

For this task, the feature branch is:

feature/login

Create the feature branch

`git checkout -b feature/login`

Developers work independently on this branch without affecting the develop or main branches.

4. Commit Changes on the Feature Branch

After implementing the login feature, save the changes.

`git add .`

`git commit -m "Added login feature"`

A commit records the current state of the project with a meaningful message.

5. Merge Feature Branch into Develop

Once the feature is completed and tested, switch to the develop branch and merge the feature branch.

`git checkout develop`

`git merge feature/login`

After merging, the login feature becomes part of the development version of the project.

6. Merge Develop into Main

After testing is completed, merge the develop branch into main.

```
git checkout main
```

```
git merge develop
```

Now the stable version contains all completed features.

7. Create a Version Tag

A Git tag marks an important version of the project, such as a software release.

```
git tag -a v1.0 -m "Release v1.0"
```

Purpose of Tags

- Marks official software releases.
- Makes it easy to identify versions.
- Helps users download a specific release.

Example:

Version : v1.0

Description : Release v1.0

8. Rebasing the Feature Branch

If the develop branch receives new commits while the feature branch is still under development, the feature branch can be updated using git rebase.

```
git checkout feature/login
```

```
git rebase develop
```

Purpose of Rebase

- Updates the feature branch with the latest changes from develop.
- Creates a cleaner and linear commit history.
- Reduces unnecessary merge commits.

Task – 4: GitHub Organizations-

SOLUTION-

Introduction

A GitHub Organization is a shared account on GitHub that allows multiple users to collaborate on projects under one centralized workspace. Organizations are mainly used by companies, educational institutions, and open-source communities to manage repositories, teams, permissions, and projects efficiently.

Unlike a personal GitHub account, an organization can have multiple owners, administrators, and members with different access levels.

What is a GitHub Organization?

A GitHub Organization is a collection of repositories managed by a group of people. It provides tools for collaboration, security, and project management. Organizations make it easier to manage multiple developers working on the same or different projects.

For example, a software company can create one organization and store all its project repositories inside it while controlling who can access each repository.

Features of GitHub Organizations

1. Centralized Repository Management

All repositories are stored under a single organization, making project management easier.

2. Team Management

Members can be grouped into teams such as Developers, Testers, and DevOps. Permissions can be assigned to each team instead of individual users.

3. Access Control

Organizations allow administrators to control who can view, edit, or manage repositories by assigning different roles.

4. Multiple Owners

An organization can have more than one owner, ensuring that project management is not dependent on a single person.

5. Collaboration

Developers can work together using branches, pull requests, code reviews, and issue tracking.

6. Security

GitHub Organizations provide better security through role-based permissions, two-factor authentication (2FA), and audit logs.

7. GitHub Actions

Organizations support GitHub Actions to automate building, testing, and deploying applications.

Roles in a GitHub Organization

Owner

- **Has full control over the organization.**
- **Can add or remove members.**
- **Manages repositories and organization settings.**

Member

- **Can access repositories based on the permissions assigned.**
- **Can contribute to projects and collaborate with other members.**

Team

- **A collection of members with the same responsibilities.**
- **Permissions are assigned to the entire team instead of each individual.**

Benefits of GitHub Organizations

- **Easy collaboration among team members.**
- **Centralized project management.**

- Better security and permission control.
- Supports multiple repositories.
- Simplifies code review through Pull Requests.
- Enables Continuous Integration and Continuous Deployment (CI/CD) using GitHub Actions.
- Suitable for companies, colleges, and open-source projects.

➡ Real-World Example

Suppose a software company named ABC Technologies develops multiple applications. Instead of storing each project in individual accounts, the company creates a GitHub Organization called ABC-Technologies.

Inside the organization:

- The Developers team writes the code.
- The Testers team verifies the software.
- The DevOps team manages deployment and automation.
- Owners manage repositories and member permissions.

This structure makes collaboration secure, organized, and efficient.

Task – 5: GitHub Collaboration Architecture-

SOLUTION-

Introduction

GitHub Collaboration Architecture is a workflow that enables multiple developers to work on the same project in an organized and secure manner. GitHub provides features such as Branch Protection Rules, Pull Requests, Issue Templates, Project Boards, and CONTRIBUTING.md to improve collaboration, maintain code quality, and efficiently manage software development.

1. Branch Protection on the Main Branch

The main branch is the most important branch of a repository because it contains the stable and production-ready version of the project. To prevent accidental or unauthorized changes, GitHub provides Branch Protection Rules.

For this task, branch protection is configured on the main branch with the following rule:

- **Require at least 1 Pull Request (PR) review before merging.**

Purpose

- **Prevents direct changes to the main branch.**
- **Ensures every change is reviewed by another team member.**
- **Maintains code quality.**
- **Reduces the chances of bugs entering the production branch.**

2. Issue Templates

GitHub provides Issue Templates to standardize the way users report bugs or request new features.

Two templates are created:

Bug Report Template

Used when users find an error in the project.

Example fields:

- **Bug title**
- **Description**
- **Steps to reproduce**
- **Expected result**
- **Actual result**
- **Screenshots (if available)**

Feature Request Template

Used when suggesting a new feature.

Example fields:

- **Feature title**
- **Description**
- **Benefits**
- **Expected implementation**

Purpose

- **Makes issue reporting consistent.**
- **Provides complete information to developers.**
- **Simplifies issue tracking.**

3. GitHub Kanban Project Board

A Kanban Board is a visual project management tool used to track the progress of tasks.

For this task, the project board contains four columns:

Backlog

Stores newly created tasks that have not yet been started.

In Progress

Contains tasks currently being worked on by developers.

Review

Contains completed tasks waiting for review or approval.

Done

Contains tasks that have been reviewed, approved, and completed.

Workflow

Backlog

|



In Progress

|



4. Sample Issues

Three sample issues are created and managed through the Kanban board.

Issue 1

Title: Fix Login Bug

Progress:

Backlog → In Progress → Review → Done

Issue 2

Title: Improve README Documentation

Progress:

Backlog → In Progress → Review → Done

Issue 3

Title: Add Dark Mode Feature

Progress:

Backlog → In Progress → Review → Done

Each issue moves through the project board until it is completed.

5. CONTRIBUTING.md File

The CONTRIBUTING.md file contains guidelines for contributors who want to participate in the project.

It explains:

- **How to create a branch**
- **Coding standards**
- **Commit message format**
- **Pull Request process**
- **Code review rules**
- **Issue reporting guidelines**

Example Workflow

1. **Fork or clone the repository.**
2. **Create a new feature branch.**
3. **Make the required changes.**
4. **Commit the changes with a meaningful message.**
5. **Push the branch to GitHub.**
6. **Create a Pull Request.**
7. **Wait for at least one reviewer to approve the changes.**
8. **Merge the Pull Request into the main branch.**
9. **Delete the feature branch after merging.**

GitHub Collaboration Workflow

Developer



Create Feature Branch



Develop Feature



Commit Changes



Push to GitHub



Create Pull Request



Code Review (1 Approval Required)



Merge into Main Branch



Delete Feature Branch

Benefits of GitHub Collaboration Architecture

- **Enables team collaboration.**
- **Protects the main branch from direct changes.**
- **Improves code quality through Pull Request reviews.**
- **Organizes tasks using a Kanban board.**
- **Standardizes issue reporting using templates.**
- **Provides contribution guidelines for developers.**
- **Supports efficient project management.**